QALight

Frontend-розробка

Заняття 21. Асинхронний код

Викладач курсу: Іван Ніконов, ТОВ «Тренінг-центр КьюЕйЛайт»

Порядок денний



Асинхронний код.
Технологія AJAX. Асинхронні події. Властивості та методи об'єкта XMLHttpRequest. Асинхронний запит. Доступ до даних.
Створення асинхронного запиту для отримання курсу обміну валют.

Асинхронний код

Технологія AJAX. Асинхронні події. Властивості та методи об'єкта XMLHttpRequest. Асинхронний запит. Доступ до даних.

JavaScript є однопотоковою мовою програмування

Теорія: AJAX

JavaScript є **однопотоковою** мовою програмування, тобто JavaScript **має один потік виконання і** може виконувати лише одне завдання за раз. Під час виконання скрипта браузер зупиняє всю іншу обробку, поки скрипт працює. Це необхідно, оскільки зміни DOM сторінки не можуть відбуватися в паралельних потоках.

Наприклад, JavaScript виявляє натискання кнопки, виконує обчислення та оновлює DOM. Після завершення браузер може обробити наступний елемент у черзі.

JavaScript може відправляти мережеві запити на сервер та підвантажувати нову інформацію за потребою. Можна використовувати мережевий запит без перезавантаження сторінки.

Окремі потоки викликають проблему, коли JavaScript викликає повільний процес, наприклад запит Ajax у браузері або операцію з базою даних на сервері. Така операція може тривати кілька секунд або навіть хвилин. Під час очікування відповіді браузер **блокувався**. На сервері програма не може обробляти подальші запити користувачів.

Асинхронний JavaScript

Теорія: AJAX

Асинхронний JavaScript - це підхід до написання коду, який дозволяє програмі виконувати тривалі завдання (наприклад, запити до сервера, читання файлів або таймери) у фоновому режимі, не блокуючи виконання іншого коду та інтерфейсу користувача.
Щоб зрозуміти це простими словами, варто порівняти синхронний та асинхронний підходи.

- Синхронний код (**Послідовний**): Виконується строго рядок за рядком. Якщо другий рядок коду робить великий запит до бази даних, який займає 5 секунд, то третій рядок буде чекати всі 5 секунд. У браузері в цей момент сторінка просто "зависає" - користувач не може нічого клікнути.
- Асинхронний код (**Паралельний/Фоновий**): Запускає тривалу операцію і відразу йде виконувати код далі. Коли фонова операція завершується, JavaScript повертається до неї, щоб обробити результат. Сторінка залишається повністю робочою та плавною.

AJAX

Теорія: AJAX

Асинхронний JavaScript і XML (**Ajax або AJAX**) - це техніка веб-розробки, за якої веб-програма отримує вміст із сервера, роблячи асинхронні запити HTTP, і використовує новий вміст для оновлення відповідних частин сторінки, не вимагаючи повного завантаження сторінки. AJAX дає можливість браузеру у фоновому режимі обмінюватись даними з веб-сервером.

Асинхронні завдання, це ті, які під час виконання **не блокують поток виконання**. Таким чином, програма може виконувати інші завдання, поки виконується асинхронне завдання. Такі завдання називають **неблокуючими**.

1. На веб-сторінці відбувається подія (сторінка завантажується, натискається кнопка)
2. Об’єкт XMLHttpRequest створюється за допомогою JavaScript
3. Об’єкт XMLHttpRequest надсилає запит на веб-сервер
4. Сервер обробляє запит
5. Сервер надсилає відповідь на веб-сторінку
6. Відповідь зчитується JavaScript
7. Належну дію (наприклад, оновлення сторінки) виконує JavaScript

Асинхронні функції

Теорія: AJAX

Функції, що виконуються паралельно з іншими функціями, називаються **асинхронними.** Замість того, щоб чекати завершення, процес викликає іншу функцію, поки результат не буде готовий. Це називається **зворотним викликом**, що передається як аргумент асинхронній функції.
Колбек (**Callback function**) - це функція, яка передається як аргумент в іншу функцію і викликається лише тоді, коли завершиться фонова дія.

```
console.log("1. Початок");

    setTimeout(function () {
        console.log("2. Минуло 2 секунди у фоні");
    }, 2000);

    console.log("3. Кінець");
    // Результат у консолі:// 1. Початок// 3. Кінець// (пауза 2 секунди)// 2. Минуло 2 секунди у фоні
```

Якщо у вас багато залежних запитів, код перетворюється на "Пекло колбеків" (**Callback Hell**), через величезну кількість вкладеностей.

Цикл подій

Теорія: AJAX

Сам по собі рушій JavaScript (наприклад, V8 у Chrome чи Node.js) є **однопотоковим**. Це означає, що він **може робити лише одну дію одночасно**.
Магія асинхронності відбувається завдяки **оточенню браузера (Web APIs)**. Коли ви викликаєте **setTimeout**, JavaScript передає це завдання самому браузеру. Браузер робить запит у своєму окремому системному потоці, а коли отримує результат - кидає сигнал у **спеціальну чергу**.

Механізм під назвою **Event Loop (Цикл подій)** постійно стежить: як тільки головний потік JavaScript звільняється, Event Loop бере завдання з черги та виконує його.

Event Loop (Цикл подій) - це механізм, який дозволяє JavaScript бути асинхронним і виконувати неблокуючі операції, попри те, що сам двигун JS є однопотоковим (може робити лише одну дію одночасно).
Він працює за принципом нескінченного циклу, який постійно перевіряє, чи вільний головний потік, і якщо вільний - бере завдання з черги та передає їх на виконання.

Процес Event Loop

Теорія: AJAX

Щоб зрозуміти Event Loop, уявіть собі конвеєр, що складається з 4 головних частин:

1. **Call Stack (Стек викликів)**: Це місце, де безпосередньо виконується код (рядок за рядком). Працює за принципом **LIFO** (останній зайшов - перший вийшов). Якщо функція виконується, вона перебуває в стеку.
2. **Web APIs (Браузерне середовище)**: Сюди JS передає складні або тривалі завдання, які сам робити не вміє (таймери setTimeout, мережеві запити fetch/XHR, обробники подій addEventListener). Вони виконуються браузером у фоні.
3. **Черги завдань (Callback Queues)**: Коли фонове завдання у Web APIs завершується, його результат (колбек) потрапляє в чергу і чекає свого часу на виконання. Насправді черг дві:

- Microtask Queue (**Event Loop**): Для промісів (.then, async/await) та MutationObserver.
- Macrotask Queue (**Черга макрозавдань або Callback Queue**): Для setTimeout, setInterval, подій кліків тощо.

4. **Event Loop (Цикл подій)**: Диспетчер, який сидить і дивиться: "Чи порожній Call Stack?". Якщо Стек повністю вільний, він переносить туди завдання з черг.

Алгоритм роботи Event Loop

Теорія: AJAX

Коли JavaScript запускається, Event Loop працює за таким чітким циклом:

1. Виконує весь синхронний код у Call Stack, поки той не стане порожнім.
2. Перевіряє Чергу мікрозавдань (Microtasks): Виконує абсолютно всі мікрозавдання, які там накопичилися. Якщо під час їх виконання з'являються нові мікрозавдання, вони додаються в кінець і теж виконуються в цьому ж циклі.
3. Оновлює рендеринг сторінки (якщо це необхідно зробити в цей момент часу).
4. Бере ОДНЕ макрозавдання (Macrotask): Переносить його з черги в Call Stack і виконує.
5. Повертається до пункту 2 (цикл повторюється).

Важливе правило пріоритету: Мікрозавдання (Promises) ЗАВЖДИ мають вищий пріоритет, ніж макрозавдання (setTimeout). Event Loop не візьметься за наступний таймаут, поки повністю не очистить чергу промісів.

Приклад роботи Event Loop

Теорія: AJAX

Розберемо класичне питання з технічних співбесід.

```
// Яким буде порядок виведення в консоль?
// Відповідь:
// 1. Синхронний старт
// 4. Синхронний кінець
// 3. Мікрозавдання (Promise)
// 2. Макрозавдання (setTimeout)

console.log('1. Синхронний старт');
setTimeout(() => {
    console.log('2. Макрозавдання (setTimeout)');
}, 0);
Promise.resolve().then(() => {
    console.log('3. Мікрозавдання (Promise)');
});

console.log('4. Синхронний кінець');
```

Як це бачить Event Loop

Теорія: AJAX

Як це бачить Event Loop:

1. console.log('1...') потрапляє в Стек і відразу виконується. (Виведено: 1)
2. setTimeout потрапляє в Стек. JS бачить, що це браузерна операція, передає її у Web APIs і видаляє з Стеку. Оскільки таймаут 0мс, Web APIs відразу перекидає колбек у Чергу макрозавдань.
3. Promise потрапляє в Стек, виконується, а його .then() перекидається в Чергу мікрозавдань.
4. console.log('4...') потрапляє в Стек і виконується. (Виведено: 4)
5. Основний синхронний код завершився. Стек порожній.
6. Час мікрозавдань: Event Loop заглядає в Чергу мікрозавдань, бачить там проміс, закидає в Стек і виконує. (Виведено: 3)
7. Черга мікрозавдань тепер порожня.
8. Час макрозавдань: Event Loop бере одне завдання з черги макрозавдань (наш таймаут), закидає в Стек і виконує. (Виведено: 2)

Правильна відповідь: 1 -> 4 -> 3 -> 2

Якщо ви напишете важкий синхронний код (наприклад, цикл на 10 мільярдів ітерацій), він повністю заблокує Call Stack. Оскільки Стек не звільниться, Event Loop не зможе взяти завдання з черги подій (користувач не зможе навіть клікнути на кнопку), а браузер видасть попередження: "Сторінка не відповідає".

Вбудований об’єкт браузера XMLHttpRequest

Теорія: AJAX

В основі технології AJAX закладено використання об’єкта **XMLHttpRequest**, який може відправляти та отримувати інформацію у форматах XML, HTML тощо.

XMLHttpRequest – це **вбудований об’єкт браузера**, який дозволяє виконувати HTTP-запити за допомогою JavaScript.

Попри те, що в назві є слово “XML”, він може працювати з будь-якими даними, а не лише у форматі XML. Ми можемо завантажувати з/на сервер файли, відстежувати прогрес та багато іншого.

XMLHttpRequest стандартизований WHATWG.
Часто використовують два формати:

- **XML** (eXtensible Markup Language);
- **JSON** (JavaScript Object Notation).

Режими роботи XMLHttpRequest

Теорія: AJAX

XMLHttpRequest має два режими роботи: синхронний і асинхронний. Асинхронний режим роботи використовується в більшості випадків.

Щоб зробити запит, нам потрібно виконати 3 кроки:

```
// Створити XMLHttpRequest:
let xhr = new XMLHttpRequest();

// Конструктор не має аргументів.

// Ініціалізувати його, зазвичай це роблять відразу після new XMLHttpRequest:
xhr.open(method, URL, [async, user, password]);
```

Метод open

Теорія: AJAX

```
xhr.open(method, URL, [async, user, password]);
```

Метод open приймає основні параметри запиту:

method – HTTP-метод. Зазвичай "GET" або "POST".

URL – URL для запиту, зазвичай це рядок, але може бути і об’єктом URL.

async – якщо явно встановлено значення false, тоді запит буде синхронним, ми розглянемо це трохи пізніше.

user, password – логін та пароль для базової HTTP аутентифікації (якщо потрібно).

Виклик метода open, всупереч його назві, не відкриває з’єднання. Він лише налаштовує запит, але мережева активність починається лише після виклику метода send.

Надіслати запит

Теорія: AJAX

xhr.send([body])

Цей метод відкриває з’єднання та надсилає запит на сервер. Необов’язковий параметр body містить тіло запиту.

HTTP-методи запиту, такі як GET, не мають тіла.
HTTP-методи POST, використовують body для завантаження даних на сервер.

Прослуховування події xhr

Теорія: AJAX

Найчастіше використовуються ці три події:

load – спрацьовує, коли запит завершено (навіть якщо статус HTTP дорівнює 400 або 500) і відповідь сервера повністю завантажено.

error – спрацьовує, коли запит не може бути виконано, наприклад мережа не працює або задана не коректна URL-адреса.

progress – періодично спрацьовує під час завантаження відповіді, та повідомляє, скільки байтів було завантажено.

```
xhr.onload = function() { alert(`Завантажено: ${xhr.status} ${xhr.response}`);  };
  xhr.onerror = function() { // спрацьовує лише в тому випадку, якщо запит взагалі неможливо зробити
    alert(`Мережева помилка`);
  };
  xhr.onprogress = function(event) { // спрацьовує періодично
    // event.loaded - скільки байтів було завантажено
    // event.lengthComputable = true, якщо сервер надіслав заголовок Content-Length
    // event.total - загальна кількість байтів (якщо сервер надіслав заголовок Content-Length)
    alert(`Отримано ${event.loaded} з ${event.total}`);
  };
```

Хід завантаження

Теорія: AJAX

Код завантажує /article/xmlhttprequest/example/load із сервера та показує хід завантаження:

```
// 1. Створюємо новий об’єкт XMLHttpRequest
 let xhr = new XMLHttpRequest();

 // 2. Налаштовуємо його: GET-запит для URL-адреси /images/
 xhr.open('GET', '/images/product7.jpg', true);

 // 3. Відправляємо запит мережею
 xhr.send();

 // 4. Код нижче буде виконано після отримання відповіді
  xhr.onerror = function () {
     alert("Не вдалося виконати запит");
 };
```

Хід завантаження

Теорія: AJAX

```
xhr.onload = function () {
    if (xhr.status != 200) { // аналізуємо HTTP-статус відповіді
        alert(`Помилка ${xhr.status}: ${xhr.statusText}`); // наприклад 404: Не знайдено
    } else { // виводимо результат
        alert(`Запит завершено, отримано ${xhr.response.length} байт`); // властивість `xhr.response` містить відповідь сервера
    }
};

xhr.onprogress = function (event) {
    if (event.lengthComputable) {
        alert(`Отримано ${event.loaded} із ${event.total} байт`);
    } else {
        alert(`Отримано ${event.loaded} байт`); // якщо сервер не надіслав заголовок Content-Length
    }
};
```

Властивості xhr

Теорія: AJAX

Після відповіді сервера, ми можемо отримати результат у таких властивостях xhr:

status
Код HTTP статусу (число): 200, 404, 403 і так далі, може бути 0 у разі помилки не пов'язаної з HTTP.
statusText
Статусне повідомлення HTTP (рядок): зазвичай OK для 200, Not Found для 404, Forbidden для 403 тощо.
response (старі скрипти можуть використовувати responseText)
Тіло відповіді сервера.

Ми також можемо вказати час очікування відповіді за допомогою відповідної властивості:
xhr.timeout = 10000; // час очікування в мс, 10 секунд

Якщо запит не завершився протягом заданого часу, він скасовується і спрацьовує подія **timeout**.

URL-адреса з параметрами

Теорія: AJAX

Щоб додати параметри до URL-адреси, наприклад ?name=value, і забезпечити належне кодування, ми можемо використовувати **об’єкт URL**:

```
let url = new URL('https://google.com/search');
url.searchParams.set('q', 'test me!');

// параметр 'q' закодований
xhr.open('GET', url); // https://google.com/search?q=test+me%21
```

Тип відповіді

Теорія: AJAX

Ми можемо встановити очікуваний формат відповіді сервера за допомогою властивості xhr.responseType:

"" (за замовчуванням) – отримати як рядок,
"text" – отримати як рядок,
"arraybuffer" – отримати як ArrayBuffer (для двійкових даних див. розділ ArrayBuffer, бінарні масиви),
"blob" – отримати як Blob (для двійкових даних див. розділ Blob),
"document" – отримати як XML-документ (може використовувати XPath та інші методи XML) або HTML-документ (на основі типу MIME для отриманих даних),
"json" – отримати як JSON (автоматичний аналіз).

Відповідь у форматі JSON

Теорія: AJAX

```
// відповідь у форматі JSON:
let xhr = new XMLHttpRequest();
xhr.open('GET', '/example/json');
xhr.responseType = 'json';
xhr.send();

// тіло відповіді {"message": "Привіт, світ!"}
xhr.onload = function() {
 let responseObj = xhr.response;
 alert(responseObj.message); // Привіт, світ!
};
```

У старих скриптах ви також можете знайти властивості xhr.responseText і навіть xhr.responseXML.

Вони існують з історичних причин, щоб отримати рядок або XML-документ. Зараз ми повинні встановлювати формат у xhr.responseType і отримувати відповідь через властивість xhr.response.

Відповіді сервера

Теорія: AJAX

Відповіді сервера міститься у властивостях xhr:

- **status**: код HTTP статусу: 200, 404, 403, може бути 0 у разі помилки не пов’язаної з HTTP.
- **statusText**: статус HTTP: зазвичай OK для 200, Not Found для 404, Forbidden для 403 тощо.
- **response**: тіло відповіді сервера.

Можна також вказати час очікування відповіді за допомогою відповідної властивості:
xhr.timeout = 10000; // час очікування в мс, 10 секунд

Якщо запит не завершився протягом заданого часу, він скасовується і спрацьовує подія **timeout**.

Можна встановити очікуваний формат відповіді сервера за допомогою властивості .responseType:

- "" (за замовчуванням) – отримати як рядок,
- "text" – отримати як рядок,
- "arraybuffer" – отримати як ArrayBuffer (для двійкових даних),
- "blob" – отримати як Blob (для двійкових даних),
- "document" – отримати як XML-документ або HTML-документ,
- "json" – отримати як JSON.

Стани запиту

Теорія: AJAX

XMLHttpRequest змінює свій стан в процесі виконання запиту. Поточний стан доступний у властивості xhr.readyState.

Список усіх станів, згідно зі специфікацією:

UNSENT = 0; // початковий стан
OPENED = 1; // викликано метод open
HEADERS_RECEIVED = 2; // отримано заголовки відповіді
LOADING = 3; // завантажується відповідь (отримано пакет даних)
DONE = 4; // запит завершено

Стани об’єкту XMLHttpRequest змінюються у порядку 0 → 1 → 2 → 3 → … → 3 → 4. Стан 3 повторюється щоразу, коли отримується пакет даних.

Події readystatechange

Теорія: AJAX

Ми можемо відстежувати події за допомогою події readystatechange:

```
xhr.onreadystatechange = function() {
 if (xhr.readyState == 3) {
   // завантаження
 }
 if (xhr.readyState == 4) {
   // запит завершено
 }
};
```

Ви можете натрапити на обробники події readystatechange у дуже старому коді. Вони там з історичних причин, оскільки раніше не було події load та інших. Нині замість цього використовують обробники подій load/error/progress.

Скасування запиту

Теорія: AJAX

Ми можемо скасувати запит у будь-який час. Це можна зробити за допомогою метода xhr.abort():

```
xhr.abort(); // скасовуємо запит
```

Це ініціює подію abort, а xhr.status стає 0.

Синхронні запити

Теорія: AJAX

Якщо в методі open для третього параметра **async** встановлено значення **false**, то запит виконується **синхронно**.
Іншими словами, виконання JavaScript призупиняється в момент виклику метода send() і відновлюється після отримання відповіді. Схожим чином поводяться команди alert та prompt.
Ось переписаний приклад, де 3-й параметр open встановлено у false:

```
let xhr = new XMLHttpRequest();
xhr.open('GET', '/hello.txt', false);

try {
 xhr.send();
 if (xhr.status != 200) {
   alert(`Error ${xhr.status}: ${xhr.statusText}`);
 } else {
   alert(xhr.response);
 }
} catch(err) { // для оброки помилок використовуємо try...catch замість події onerror
 alert("Не вдалося виконати запит");
}
```

Синхронні запити використовуються дуже рідко

Теорія: AJAX

Можливо це і виглядає прийнятно, але синхронні запити використовуються рідко, оскільки вони блокують виконання JavaScript на сторінці до моменту завершення завантаження. У деяких браузерах прокручування стає неможливим. Якщо синхронний запит займає занадто багато часу, браузер може запропонувати закрити «висячу» веб-сторінку.

Багато розширених можливостей XMLHttpRequest, таких як запит до іншого домену або встановлення часу очікування відповіді, недоступні для синхронних запитів. Крім того, як бачите, немає індикаторів прогресу.

Через все це синхронні запити використовуються дуже рідко, майже ніколи.

HTTP-заголовки

Теорія: AJAX

XMLHttpRequest дозволяє як надсилати власні заголовки, так і читати їх з відповіді.
Існує 3 методи для роботи з HTTP-заголовками:

setRequestHeader(name, value)

Встановлює заголовок запиту із заданими ім’ям name та значенням value.

Наприклад:

```
xhr.setRequestHeader('Content-Type', 'application/json');
```

Обмеження заголовків

Теорія: AJAX

Декількома заголовками керує виключно браузер, наприклад **Referer і Host**.

XMLHttpRequest не має права їх змінювати з метою безпеки користувача та коректності запиту. Видалити заголовок неможливо.

Ще одна особливість XMLHttpRequest полягає в тому, що неможливо скасувати setRequestHeader.

Після того, як заголовок встановлений, його не можна видалити. Додаткові виклики метода додають інформацію в заголовок, а не перезаписують її. Наприклад:

```
xhr.setRequestHeader('X-Auth', '123');
  xhr.setRequestHeader('X-Auth', '456');


  // заголовок буде таким:
  // X-Auth: 123, 456
```

Отримання заголовків

Теорія: AJAX

getResponseHeader(name)
Отримує заголовок відповіді за заданим ім’ям name (крім Set-Cookie і Set-Cookie2).

Наприклад:

```
xhr.getResponseHeader('Content-Type')
```

getAllResponseHeaders()
Повертає всі заголовки відповіді, крім Set-Cookie і Set-Cookie2.

Заголовки повертаються у вигляді єдиного рядка, наприклад:

```
Cache-Control: max-age=31536000
Content-Length: 4260
Content-Type: image/png
Date: Sat, 08 Sep 2012 16:53:16 GMT
```

Отримати об’єкт з парами ім’я/значення

Теорія: AJAX

Між заголовками завжди є розрив рядка "\r\n" (не залежить від ОС), тому ми можемо легко розділити його на окремі заголовки. Роздільником між ім’ям і значенням заголовка завжди є двокрапка з пробілом ": ". Це зафіксовано в специфікації.

Отже, якщо ми хочемо отримати об’єкт з парами ім’я/значення, нам потрібно додати трохи JS.

Ось так (припускаючи, що якщо два заголовки мають однакову назву, то останній замінює попередній):

```
 xhr.getResponseHeader('Content-Type')
let headers = xhr
    .getAllResponseHeaders()
    .split('\r\n')
    .reduce((result, current) => {
      let [name, value] = current.split(': ');
      result[name] = value;
      return result;
    }, {});

  // headers['Content-Type'] = 'image/png'
```

Об’єкт FormData

Теорія: AJAX

```
<form name="person">
  <input name="name" value="John">
  <input name="surname" value="Smith">
</form>
```

Форма надсилається з кодуванням multipart/form-data.

```
<script>
  // заповнюємо FormData даними із форми
  let formData = new FormData(document.forms.person);

  // додаємо ще одне поле
  formData.append("middle", "Lee");

  // відправляємо запит
  let xhr = new XMLHttpRequest();
  xhr.open("POST", "/post/user");
  xhr.send(formData);
  xhr.onload = () => alert(xhr.response);
</script>
```

Заголовок Content-Type: application/json

Якщо нам більше подобається формат JSON, тоді використовуємо **JSON.stringify** і надсилаємо дані як рядок. Тільки не забудьте встановити заголовок **Content-Type: application/json**, багато серверних фреймворків автоматично декодують JSON за його наявності:

```
let json = JSON.stringify({
 name: "John",
 surname: "Smith"
});

xhr.open("POST", '/submit')
xhr.setRequestHeader('Content-type', 'application/json; charset=utf-8');

xhr.send(json);
```

Метод .send(body) може надсилати майже будь-які дані у body, включаючи об’єкти Blob та BufferSource.

Об’єктна нотація JavaScript (JSON)

Теорія: AJAX

Об’єктна нотація JavaScript (JSON) - це текстовий формат для представлення структурованих даних на основі синтаксису об’єктів JavaScript. Він зазвичай використовується для передачі даних у веб-додатках (наприклад, надсилання деяких даних із сервера до клієнта, або навпаки).

JavaScript надає глобальний об’єкт JSON, який має методи для перетворення JSON-рядка на об’єкт та об’єкта на JSON-рядок. Перетворення рядка на об’єкт називається **десеріалізацією**, тоді як перетворення об’єкта на рядок, щоб його можна було передавати через мережу, називається **серіалізацією**.

Рядок JSON можна зберігати у власному файлі, який є текстовим файлом із розширенням **.json** і типом **MIME application/json**.

Існують онлайн інструменти для перетворення об’єктів на рядок і навпаки, наприклад, **JSON Formatter**: https://www.freeformatter.com/json-formatter.html

Формат JSON

Теорія: AJAX

Формат JSON майже ідентичний об’єктам JavaScript: Дані JSON записуються як пари ключ/значення (у подвійних лапках). Дані відокремлюються комами. Фігурні дужки містять об’єкти. Квадратні дужки містять масиви.
Ключі мають бути **рядками в подвійних лапках а** значення мають бути одним із типів даних: string, number, object, array, boolean, null. Наприклад: "name": "Cool Cat",

- Числа мають бути цілими чи з плаваючою комою: {"age":30}
- Значення можуть бути об’єктами: {"employee":{"name":"John", "age":30, "city":"New York"}}
- Значення в JSON можуть бути масивами: {"employees":["John", "Anna", "Peter"]}
- Значення в JSON можуть бути true/false: {"sale":true}
- Значення в JSON можуть бути null / нуль: {"middlename":null}

Під час надсилання даних на веб-сервер дані мають бути рядком. Перетворення об’єкта JavaScript на рядок здійснюється за допомогою **JSON.stringify()**.
При отриманні даних із веб-сервера дані завжди є рядком. Використовуючи метод **JSON.parse()**, отримані дані можна перетворити на об’єкти JavaScript.
https://jsonplaceholder.typicode.com/

Подія progress

Теорія: AJAX

Подія **progress** спрацьовує лише на етапі завантаження даних з сервера. Тобто якщо ми відправляємо POST запит, то XMLHttpRequest спочатку завантажує наші дані (тіло запиту) на сервер, а потім завантажує відповідь з сервера.
Якщо ми завантажуємо на сервер щось велике, то ми, звичайно, більше зацікавлені у відстеженні прогресу завантаження на сервер. Але xhr.onprogress тут нам не допоможе.

Існує ще один об’єкт без методів, призначений виключно для відстеження подій завантаження на сервер: xhr.upload.
Він генерує події, подібні до xhr, але xhr.upload ініціює їх виключно під час завантаження на сервер:

loadstart – розпочато завантаження на сервер.
progress – спрацьовує періодично під час завантаження.
abort – завантаження перервано.
error – помилка не пов’язана з HTTP.
load – завантаження успішно завершено.
timeout – час очікування завантаження минув (якщо встановлено властивість timeout).
loadend – завантаження завершено (успішно або з помилкою).

Обробник onprogress

Теорія: AJAX

```
xhr.upload.onprogress = function(event) {
 alert(`Завантажено на сервер ${event.loaded} із ${event.total} байтів`);
};

xhr.upload.onload = function() {
 alert(`Завантаження на сервер успішно завершено.`);
};

xhr.upload.onerror = function() {
 alert(`Сталася помилка під час завантаження: ${xhr.status}`);
};
```

Завантаження файлу на сервер з індикацією прогресу

Теорія: AJAX

```
// завантаження файлу на сервер з індикацією прогресу:
<input type="file" onchange="upload(this.files[0])">
function upload(file) {
 let xhr = new XMLHttpRequest();
 // відстежуємо хід завантаження на сервер
 xhr.upload.onprogress = function(event) {
   console.log(`Завантажено ${event.loaded} із ${event.total}`);
 };
 // відстежуємо завершення: успішне чи ні
 xhr.onloadend = function() {
   if (xhr.status == 200) {
     console.log("успішно");
   } else {
     console.log("помилка " + this.status);
   }
 };
 xhr.open("POST", "/article/xmlhttprequest/post/upload");
 xhr.send(file);
```

Запити на інші джерела

Теорія: AJAX

XMLHttpRequest може робити запити іншим джерелам використовуючи політику CORS.
Він за замовчуванням не надсилає іншим джерелам заголовки HTTP-авторизації та cookie.

Щоб увімкнути їх, встановіть для xhr.withCredentials значення true:

```
let xhr = new XMLHttpRequest();
xhr.withCredentials = true;


xhr.open('POST', 'http://anywhere.com/request');
```

Події

Теорія: AJAX

loadstart – запит почався.
progress – надійшов пакет даних відповіді, все тіло відповіді на даний момент знаходиться у властивості response.
abort – запит було скасовано викликом xhr.abort().
error – сталася помилка підключення, наприклад неправильне доменне ім’я. Не спрацьовує для HTTP-помилок, таких як 404.
load – запит успішно завершено.
timeout – запит було скасовано через тайм-аут (тільки якщо він був встановлений).
loadend – спрацьовує після подій load, error, timeout або abort.

Події error, abort, timeout, та load є взаємовиключними. Може спрацювати лише одна з них.

Найчастіше використовувані події – це завершення завантаження (load), помилка завантаження (error), або ми можемо використовувати один обробник loadend і перевірити властивості об’єкта запиту xhr, щоб побачити, що сталося.

Подія onload

Теорія: AJAX

За допомогою об’єкта XMLHttpRequest можна визначити функцію зворотного виклику, яка буде виконуватись, коли об’єкт запиту отримує відповідь:

```
xhr.onload = function() {
 document.getElementById("demo").innerHTML = this.responseText;
}

xhr.open("GET", "ajax_info.txt");
xhr.send();
```

Асинхронний код.

Створення асинхронного запиту для отримання курсу обміну валют.

Створення запиту

Практика: AJAX

```
    <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.4.1/css/bootstrap.min.css"
integrity="sha384-Vkoo8x4CGsO3+Hhxv8T/Q5PaXtkKtu6ug5TOeNV6gBiFeWPGFN9MuhOf23Q9Ifjh"
crossorigin="anonymous">
  <div class="card text-center">
      <div class="card-header"> Hello, AJAX! </div>
      <div class="card-body">
          <h4 class="card-title">Console output:</h4>
          <p class="card-text" id="console">Processing server response.</p>
          <a href="#" class="btn btn-primary getBtn">Run</a>
      </div>
      <div class="card-footer text-muted" id="printResult">server response</div>
  </div>

  <!-- Optional JavaScript -->
  <script src="./xhr.js"></script>
```

Створення запиту

Практика: AJAX

```
function CreateRequest() {
   let httpRequest = false;
   if (window.XMLHttpRequest) {  // Gecko-copability, Safari, Konqueror
       httpRequest = new XMLHttpRequest();
       httpRequest.overrideMimeType("text/plain; charset=x-user-defined");
   } else if (window.ActiveXObject) {
       try {
           httpRequest = new ActiveXObject("Microsoft.XMLHTTP");
       } catch (CatchException) {
           httpRequest = new ActiveXObject("Msxml2.XMLHTTP");
       }
   }
   if (!httpRequest) { console.log("Don`t create XMLHttpRequest"); }
   return httpRequest;
}
```

Створення запиту

Практика: AJAX

```
function el(selector) {return document.querySelector(selector);}
 function myConsole(c, status='') {
   let message, alertClass, text;
   switch (c) {
     case 1:
        message = '1: Preparation for sending...';
        alertClass = 'alert-primary';
        break;
     case 2:
        message = '2: Sent...';
        alertClass = 'alert-primary';
        break;
     case 3:
        message = '3: Processing exchange...';
        alertClass = 'alert-warning';
        break;
```

Створення запиту

Практика: AJAX

```
    case 4:
        message = '4: Exchange completed!';
        alertClass = 'alert-success';
        break;
    case 5:
        message = 'Error: requested script not found!';
        alertClass = 'alert-danger';
        break;
    case 6:
        message = 'Error: server returned status: ' + status;
        alertClass = 'alert-danger';
        break;
    Default:  break;
}
text = `<div class="alert ${alertClass}" role="alert">${message}</div>`;
el("#console").innerHTML += text; }
```

Створення запиту

Практика: AJAX

```
function sendRequest() {
   // Create request: calling the CreateRequest function;
   const request = CreateRequest();
   const url = 'https://jsonplaceholder.typicode.com/posts/1';
   if (!request) {   // Check for request existence
      console.log("Don`t create XMLHttpRequest");
   } else {
      console.log("Ye! We created XMLHttpRequest. What to do with it?");
       request.onreadystatechange = function () {
          switch (request.readyState) {
             case 1:
                myConsole(1);  break;
             case 2:
                myConsole(2);  break;
             case 3:
                myConsole(3);  break;
```

Створення запиту

Практика: AJAX

```
            case 4:
                {
                    if (request.status == 200) {
                        myConsole(4);
                        el("#printResult").innerHTML = "<strong>" + request.responseText + "</strong>";
                    } else if (request.status == 404) {
                        myConsole(5);
                    } else {
                        myConsole(6, request.status);
                    }
                    break;
                }
        } };
    request.open('GET', url, true);
    request.send('');
  }
}
el('.getBtn').addEventListener('click', sendRequest);
```

Запит для отримання курсу валют

Практика: AJAX

```
function getRateWithXHR() {
        const url = 'https://bank.gov.ua';

        // 1. Створюємо новий об'єкт запиту
        const xhr = new XMLHttpRequest();

        // 2. Ініціалізуємо запит (вказуємо метод і URL)
        xhr.open('GET', url, true); // true означає асинхронність
```

Обробник події зміни стану запиту

Практика: AJAX

```
// 3. Налаштовуємо обробник події зміни стану запиту
xhr.onload = function () {
    // Перевіряємо статус відповіді (200 OK)
    if (xhr.status >= 200 && xhr.status < 300) {
        // XHR повертає текст, тому парсити JSON потрібно вручну
        const data = JSON.parse(xhr.responseText);
        console.log('Дані через XHR:', data);
    } else {
        console.error(`Помилка сервера. Статус: ${xhr.status}`);
    }
};
```

Обробка помилок мережі

Практика: AJAX

```
        // 4. Обробка помилок мережі (наприклад, зник інтернет)
        xhr.onerror = function () {
            console.error('Помилка мережі (запит не вдався)');
        };

        // 5. Відправляємо запит
        xhr.send();
    }

    getRateWithXHR();

    // XMLHttpRequest змушує працювати з колбеками (onload, onerror), вимагає ручного виклику JSON.parse() та написання більшої кількості шаблонного коду.
```

ВПРАВИ



Вправи до цього заняття знаходяться за посиланням
https://github.com/coachjanus/frontend-dev

Посилання на довідники



Консорціум Всесвітнього павутиння https://www.w3.org/

ECMAScript https://ru.wikipedia.org/wiki/ECMAScript

ECMA http://www.ecma-international.org/

Standard ECMA-262 http://www.ecma-international.org/publications/standards/Ecma-262.htm

JavaScript Web APIs:

- https://www.w3.org/standards/webdesign/script
- https://developer.mozilla.org/en-US/docs/Glossary/JavaScript

Дякую за увагу!

До наступної зустрічі!

QALight

www.qalight.ua
info@qalight.ua